

Lab 1: The Internet Protocols

Wireshark capture and analysis of HTTP, TCP/IP, and UDP traffic

Student Name: Rawan Alanezi Student ID:431201729 Section: 5527

1. Objective

This lab aimed to use Wireshark to capture network packets and examine how HTTP, TCP, and UDP traffic appear in a real network connection. I used my Wi-Fi interface for the capture and recorded the values that appeared on my own device.

2. Part 1 – Capturing HTTP Traffic

I started a capture on the Wi-Fi interface, opened <http://neverssl.com/>, waited for the page to load, and then stopped the capture. I used the **http** display filter to view only HTTP packets.

HTTP observation

- Request method observed: GET
- URL observed: <http://wonderousglowinggoldrain.neverssl.com/online>
- Host observed: wonderousglowinggoldrain.neverssl.com
- The first request to /online received HTTP/1.1 301 Moved Permanently.
- After the redirect, another request to /online/ received HTTP/1.1 200 OK.

My observation: The browser did not receive the final page immediately. It was redirected first, and the next request returned a successful 200 OK response.

The image shows a Wireshark capture of HTTP traffic. The top pane displays a list of packets, with the first packet being a GET request for /online. The second packet is a 301 Moved Permanently response. The third packet is a GET request for /online/. The fourth packet is a 200 OK response. The bottom pane shows the details of the selected packet, which is the 200 OK response. The details pane shows the HTTP headers, including the status line: 200 OK (text/html). The body of the response is a PNG image.

No.	Time	Source	Destination	Protocol	Length	Info
1	38.608197000	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	597	GET /online HTTP/1.1
2	38.911039900	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	655	HTTP/1.1 301 Moved Permanently (text/html)
3	38.925492800	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	598	GET /online/ HTTP/1.1
4	39.179153700	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	161	HTTP/1.1 200 OK (text/html)
5	39.325475500	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	532	GET /favicon.ico HTTP/1.1
6	39.596587700	2001:16a3:eb8:9900	2001:16a3:37c:1400	HTTP	480	HTTP/1.1 200 OK (PNG)

```
22 02 71 cb 86 05 98 43 Fa 9a 36 20 86 dd 60 04 "q...C...6...
1f a2 02 1f 05 48 20 01 16 a3 0a b8 99 00 5d d7 .....@.....]
39 b5 9e 76 9f 4f 26 00 1f 13 05 7c 14 00 ba 21 9...vO@...-...I
71 65 5f c7 73 6e dd 95 00 50 a7 7b ef 32 b3 28 qe...sn...-P...{2(
b2 02 50 18 00 ff 11 21 00 00 47 45 54 20 2f 6f -P...-I-GET/o
6e 6c 69 6e 65 20 48 54 54 50 2f 31 2e 31 0d 0a nline HT TP/1.1
48 6f 73 74 3a 28 77 6f 6e 64 65 72 6f 75 73 67 Host: wo nderousg
6c 6f 77 69 6e 67 6f 6c 64 72 61 69 6e 2e 6e 65 lowinggol drain.ne
76 65 72 73 73 6c 2a 63 6f 6d 0d 0a 43 6f 6e 6e verssl.c om-Conn
65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 ection: keep-ali
76 65 0d 0a 55 78 67 72 61 64 65 2d 49 6e 73 65 ve: Upgr ade-Inse
63 75 72 65 2d 52 65 71 75 65 73 74 73 3a 20 31 cure-Req uests: 1
0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f -User-A gent: No
7a 69 6c 6c 61 2f 35 2e 30 20 28 57 69 6e 64 6f zilla/5. 0 (Mindo
77 73 20 4e 54 20 31 30 2e 30 30 20 57 69 6e 36 ws NT 10 .0; Min6
34 30 20 78 36 34 29 20 41 70 70 6c 65 57 65 62 4; x64) AppleWeb
4b 69 74 2f 35 33 37 2e 33 36 20 28 4b 48 54 4d Kit/537. 36 (KHTML
4c 2c 20 6c 69 6b 65 20 47 65 63 6b 6f 29 20 43 L, like Gecko) C
```

id (4776 bits) on interFace \Device\NPF_{D61C277A-A696-4E81-A858-712670E66D97}, id 0 <
Src: Intel_9a:36:20 (98:43:fa:9a:36:20), Dst: 22:02:71:cb:86:05 (22:02:71:cb:86:05) <
2001:16a3:eb8:9900:5dd7:39b5:9e76:9f4f, Dst: 2001:16a3:37c:1400:ba21:7165:5fc7:736e <
Transmission Control Protocol, Src Port: 56725, Dst Port: 80, Seq: 1, Ack: 1, Len: 523 <
Hypertext Transfer Protocol <
GET /online HTTP/1.1\r\n <
Host: wonderousglowinggoldrain.neverssl.com\r\n <
Connection: keep-alive\r\n <
Upgrade-Insecure-Requests: 1\r\n <
ebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0\r\n <
s/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n <
Referer: http://neverssl.com/\r\n <
Accept-Encoding: gzip, deflate\r\n <
Accept-Language: ar,en;q=0.9,en-GB;q=0.8,en-US;q=0.7\r\n <

3. Part 2 – Analyzing TCP/IP Traffic

I changed the display filter to **tcp**, selected the TCP traffic related to the NeverSSL request, and used **Follow → TCP Stream**. In my capture, Wireshark identified this conversation as stream 18.

3.1 TCP Three-Way Handshake

The connection was established with SYN, SYN-ACK, and ACK packets. The values below are from my capture:

Frame	Packet	Sequence No.	Acknowledgment No.
4357	SYN	0	0
4450	SYN, ACK	0	1
4451	ACK	1	1

Extra observation: A few SYN retransmissions appeared before the server replied, but the connection was still established successfully.

Wireshark packet capture showing a TCP three-way handshake. The display filter is `frame.number == 4357 || frame.number == 4450 || frame.number == 4451`. The packet list shows three packets:

No.	Time	Source	Destination	Protocol	Length	Info
4357	35.328514900	2001:16a3:eb8:9900	2600:1f13:37c:1400	TCP	86	Seq=0 Win=65535 Len=0 MSS=1432 WS=256 SACK_PERM [SYN] 80 → 56725
4450	38.603047500	2600:1f13:37c:1400	2001:16a3:eb8:9900	TCP	86	Seq=0 Ack=1 Min=26883 Len=0 MSS=1432 SACK_PERM WS=128 [SYN, ACK] 56725 → 80
4451	38.603263200	2001:16a3:eb8:9900	2600:1f13:37c:1400	TCP	74	Seq=1 Ack=1 Min=65280 Len=0 [ACK] 80 → 56725

The packet details for the selected packet (4450) show the TCP header with Seq=0, Ack=1, and Win=65535. The packet bytes show the raw data of the SYN-ACK packet.

3.2 TCP Data Transfer

After the handshake, I filtered the same TCP stream to show packets carrying data. The stream included the HTTP request and the server responses.

- GET /online HTTP/1.1
- HTTP/1.1 301 Moved Permanently
- GET /online/ HTTP/1.1
- HTTP/1.1 200 OK (text/html)

My observation: The client and server exchanged application data only after the TCP connection had been established.

The image shows a Wireshark packet capture of an HTTP transaction. The top pane displays a list of packets, with packet 10 selected. The middle pane shows the packet details, including the Ethernet II header, Internet Protocol Version 4 header, and Hypertext Transfer Protocol header. The bottom pane shows the raw packet data in hexadecimal and ASCII.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
10	38.686197000	2001:16a3:eb8:9900	2600:1f13:37c:1400	HTTP	597	GET /online HTTP/1.1 597
11	38.911839900	2600:1f13:37c:1400	2001:16a3:eb8:9900	HTTP	655	HTTP/1.1 301 Moved Permanently (text/html) 655
12	38.925492800	2001:16a3:eb8:9900	2600:1f13:37c:1400	HTTP	598	GET /online/ HTTP/1.1 598
13	39.178824900	2600:1f13:37c:1400	2001:16a3:eb8:9900	TCP	161	TCP [ACK] 56725 → 80 1506
14	39.179153700	2600:1f13:37c:1400	2001:16a3:eb8:9900	HTTP	161	HTTP/1.1 200 OK (text/html) 161
15	39.325475500	2001:16a3:eb8:9900	2600:1f13:37c:1400	HTTP	532	GET /favicon.ico HTTP/1.1 532
16	39.596587700	2600:1f13:37c:1400	2001:16a3:eb8:9900	HTTP	498	HTTP/1.1 200 OK (PNG) 498

Packet 10 Details:

- Ethernet II, Src: Intel_9a:36:20 (98:43:fa:9a:36:20), Dst: 22:02:71:cb:86:05 (22:02:71:cb:86:05)
- Internet Protocol Version 4, Src: 2001:16a3:eb8:9900:5dd7:39b5:9a76:9f4f, Dst: 2600:1f13:37c:1400:ba21:7165:5fc7:736e
- Hypertext Transfer Protocol, GET /online HTTP/1.1
- Host: wonderfulglowinggoldrain.neverssl.com
- Connection: keep-alive
- Upgrade-Insecure-Requests: 1
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
- Referer: http://neverssl.com/
- Accept-Encoding: gzip, deflate
- Accept-Language: ar,en;q=0.9,en-US;q=0.7

Raw Data:

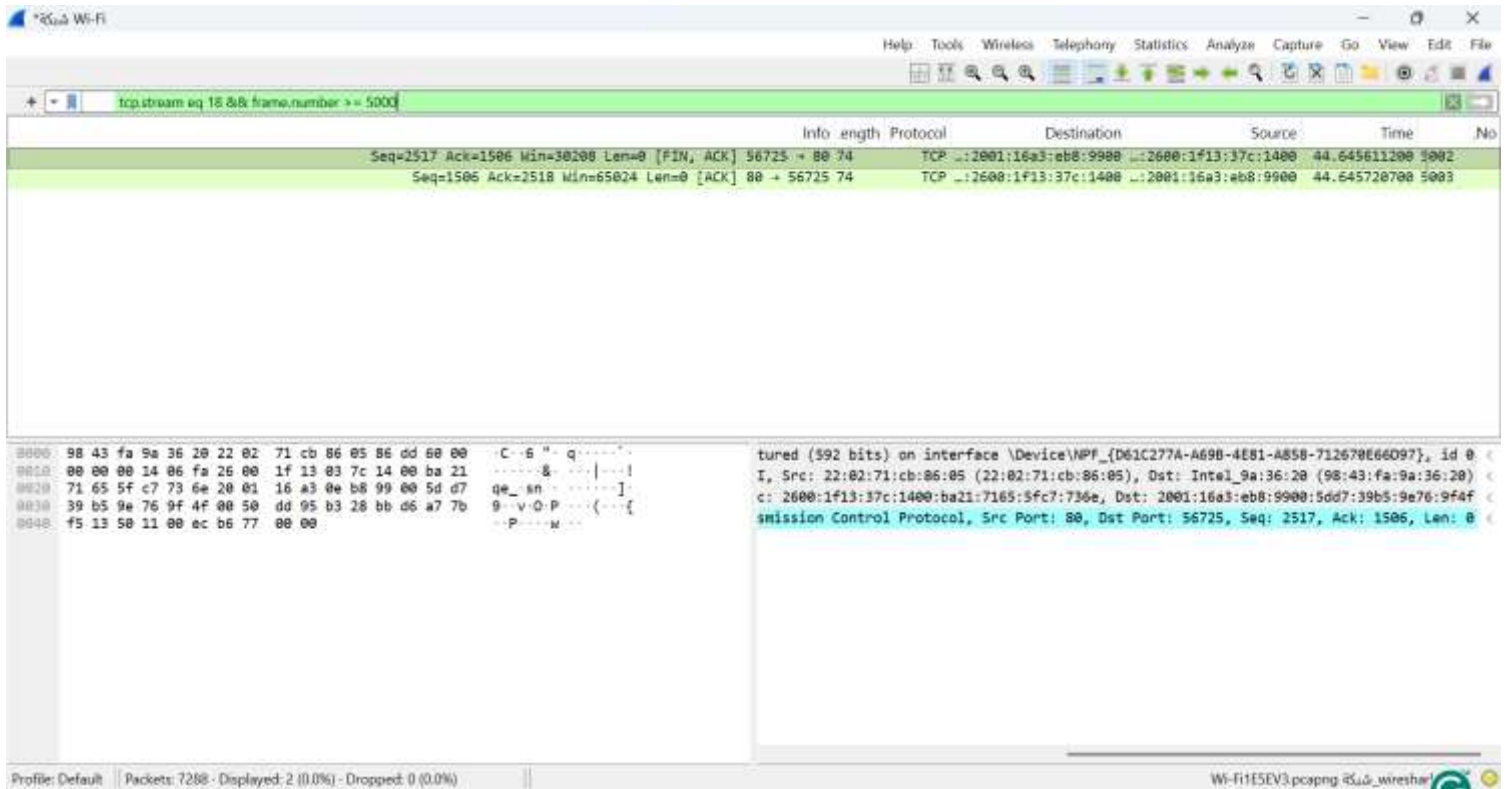
```
0000  22 02 71 cb 86 05 98 43 fa 9a 36 20 86 dd 60 04  "q...C..G...  
0010  1f a2 02 1f 06 40 20 01 16 a3 0e b8 99 00 5d d7  ....@.....]  
0020  39 b5 9e 76 9f 4f 26 00 1f 13 03 7c 14 00 ba 21  9..v.O&....|  
0030  71 65 5f c7 73 6e dd 95 00 50 a7 7b ef 32 b3 28  qe.sn...P.(-2:  
0040  b2 02 50 18 00 ff 11 21 00 00 47 45 54 20 2f 6f  .P....GET/o  
0050  6e 6c 69 6e 65 20 48 54 54 50 2f 31 2e 31 0d 0a  nline HT TP/1.1  
0060  48 6f 73 74 3a 20 77 6f 6a 64 65 72 6f 75 73 67  Host: wo nderousg  
0070  6c 6f 77 69 6a 67 6f 6c 64 72 61 69 6a 2e 6e 65  lowingol drain.ne  
0080  76 65 72 73 73 6c 2e 63 6f 6d 0d 0a 43 6f 6e 6e  verssl.c om Conn  
0090  65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69  ection: keep-ali  
00a0  76 65 0d 0a 55 70 67 72 61 64 65 2d 49 6e 73 65  ve Upgr ade-Inse  
00b0  63 75 72 65 2d 52 65 71 75 65 73 74 73 3a 20 31  cure+Req uests: 1  
00c0  0d 0a 55 73 65 72 2d 41 67 65 6a 74 3a 20 4d 6f  .User-A gent: Mo  
00d0  7a 69 6c 6c 61 2f 35 2e 30 20 28 57 69 6e 64 6f  zilla/S. 0 (Windo  
00e0  77 73 20 4e 54 20 31 30 2e 30 3b 20 57 69 6e 64 6f  ws NT 10 .0; Win6  
00f0  34 3b 20 78 36 34 29 20 41 70 70 6c 65 57 65 62  4; x64) AppleWeb  
0100  4b 69 74 2f 35 33 37 2e 33 36 20 28 4b 48 54 4d  Kit/537. 36 (KHTM  
0110  4c 2c 20 6c 69 6b 65 20 47 65 63 6b 6f 29 20 43  L, like Gecko) C
```

3.3 TCP Connection Termination

At the end of the same TCP stream, I found the packets used to close the connection.

- Frame 5002: FIN, ACK – Seq=2517, Ack=1506
- Frame 5003: ACK – Seq=1506, Ack=2518

My observation: The FIN flag indicated that one side was ready to close the connection, and the following ACK confirmed the termination request.



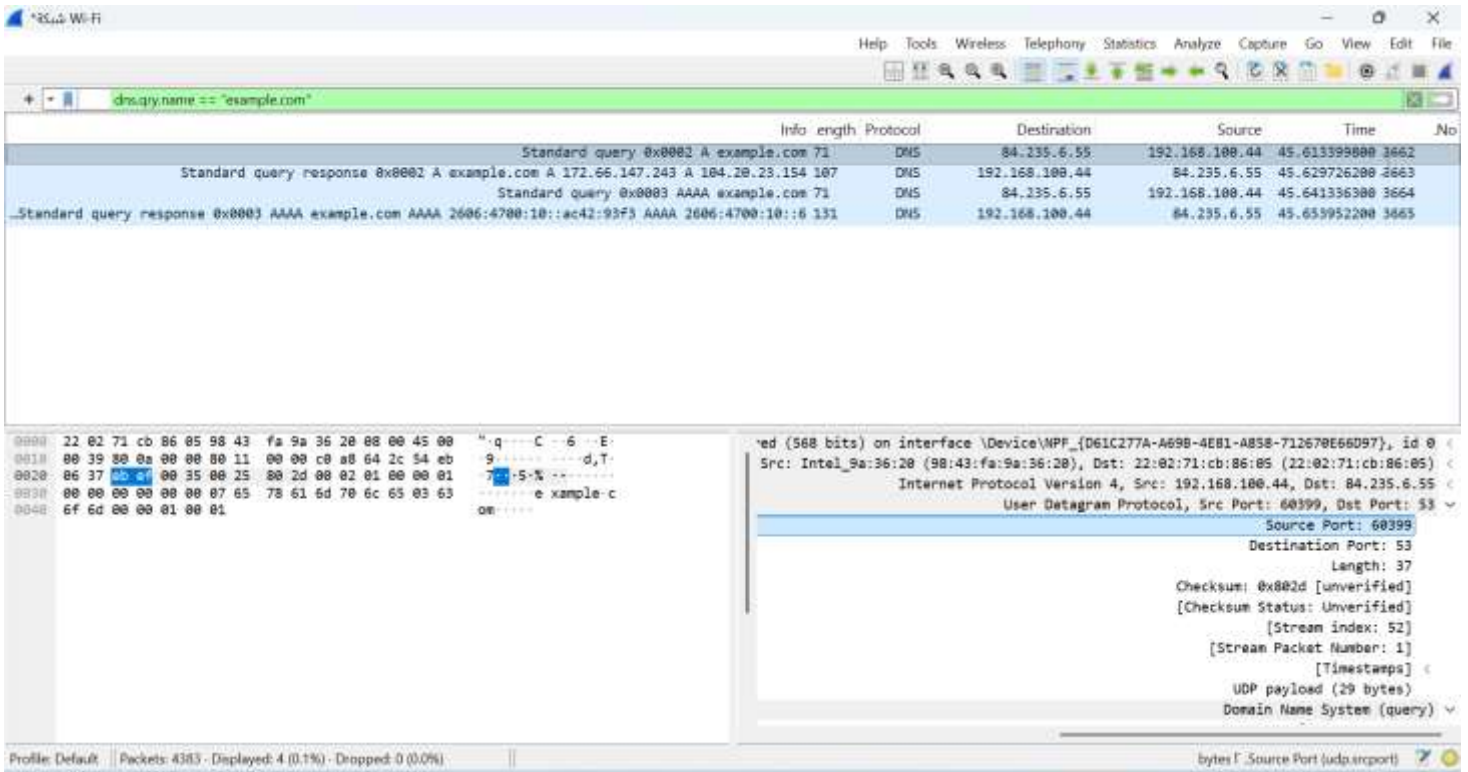
4. Part 3 – Capturing and Analyzing UDP Traffic

For UDP traffic, I started a new Wireshark capture and used nslookup example.com to generate a DNS request. After stopping the capture, I first used the udp display filter, then narrowed the results to the DNS query for example.com.

UDP packet values

Field	Observed value
Source Port	60399
Destination Port	53
UDP Length	37 bytes
UDP Payload	29 bytes
Data	DNS standard query for example.com

UDP header observation: The UDP header was simpler than the TCP header. It mainly showed the source port, destination port, length, and checksum, while TCP included extra fields for sequencing, acknowledgments, flags, and connection control.



5. Part 4 – Comparing TCP and UDP

Task 1 – Reliability, connection, integrity, and ordering

Item	TCP or UDP	Reason
Reliability and Connection Establishment	TCP	TCP establishes a connection before sending data and uses acknowledgments to confirm delivery.
Data Integrity and Ordering	TCP	TCP uses sequence numbers to keep data in order and can retransmit data when packets are missing.

Task 2 – Use cases and performance

	TCP	UDP
Use cases	Web browsing, email, and file transfer	DNS, live streaming, online games, and voice calls
Performance	More overhead because TCP maintains a connection and checks delivery.	Lower overhead and usually faster because UDP sends data without connection setup or delivery acknowledgments.

6. Conclusion

This lab helped me see the difference between the protocols directly in Wireshark instead of only reading about them. HTTP showed the browser requests and server responses, TCP showed the connection setup, data exchange, and termination, and UDP showed a smaller header with less connection-management information. The packet values in this report came from my own capture.